

SLIM-ARC:

面向端侧 Agent 的 内存感知 MoE 大模型推理协同优化

中山大学 计算机学院

依托Agent答辩OS队

<https://slim.nexa-lang.com/>

➤ 队伍信息：依托**Agent**答辩OS (T2026105589911358)

➤ 队员：欧阳易芄 马福泉 刘昊

➤ 中山大学计算机学院

➤ 指导老师：赵帅 （副教授，博士生导师）

张献伟（副教授，博士生导师）

➤ 赛题信息：

➤ **Proj59**：内存受限环境的大语言模型推理优化问题

➤ <https://slim.nexa-lang.com/>

➤ 赛题老师：宫晓利



目录 Catalogue

- ❖ 1 背景与动机 Background & Motivation
- ❖ 2 项目简述 Project Introduction
- ❖ 3 核心设计 SLIM-ARC Core Design
- ❖ 4 工程实现 Projrct Implementation
- ❖ 5 测试与评估 Testing & Evaluation
- ❖ 6 总结与展望 Summary & Outlooking

1 背景与动机

端侧Agent运行挑战
与当前主流方案存在的问题

端侧“内存墙”挑战

➤ 严峻的供需矛盾

➤ 80B参数量化大模型（Qwen3-Next-80B）体积高达**45GB**

➤ 完整运行需要 **140GB**

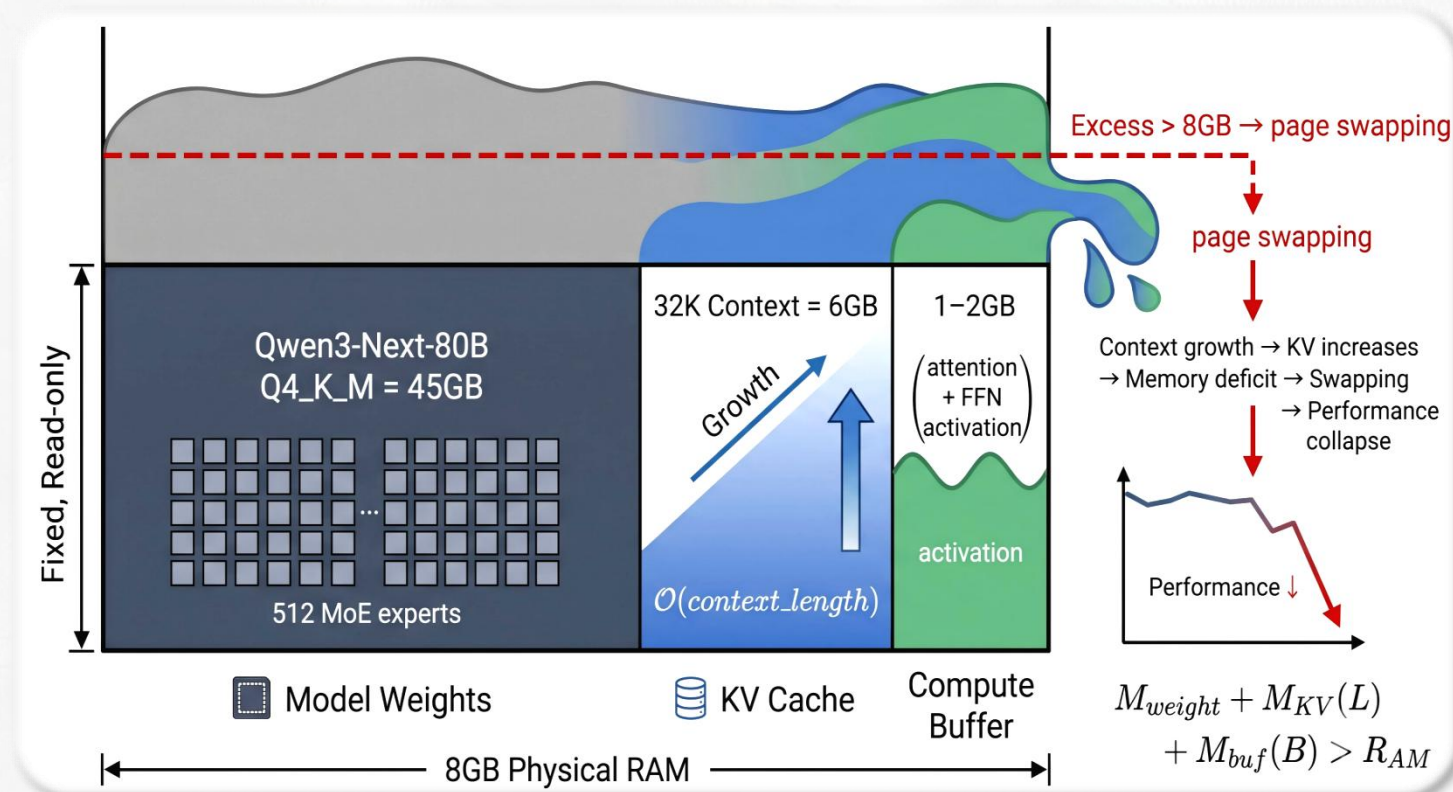
➤ 典型端侧设备物理内存仅受限于 **8至16GB**

➤ 内存消耗的“三座大山”

➤ 模型权重

➤ KV Cache

➤ 计算缓冲区



现有研究的局限性与痛点

- 高侵入性与架构兼容壁垒：
 - 代价过高：依赖 **用户态** I/O 重写（FlexInfer），或需修改底层模型结构（PowerInfer）
 - 难以落地：与主流框架存储格式冲突（MobileMoE 无法兼容 llama.cpp 合并存储的3D张量结构）
- OS默认机制触发内存溢出（OOM）：
 - 缺页调度僵化：框架默认的加载提示（WILLNEED）会触发 Linux 内核无差别的 **“顺序预读”**（Sequential Readahead）。
 - 物理内存挤兑：冗余预读瞬间耗尽页缓存（Page Cache），导致 45GB 模型在 8GB 设备的受限空间内直接触发OOM。
- 单点优化 **缺乏全局统筹**：权重按需加载、KV Cache 换页与算子优化等技术彼此解耦

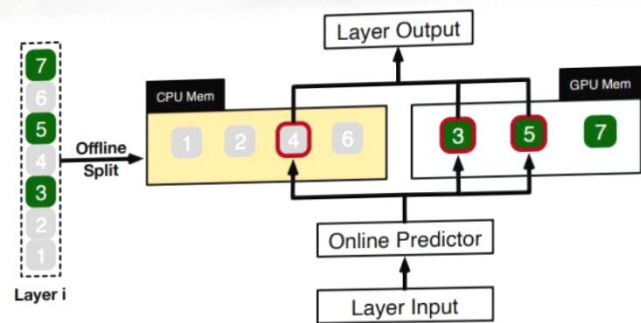


Figure 8: An illustrative example shows how PowerInfer calculates different neurons for one LLM layer.

4.2 Single Layer Example

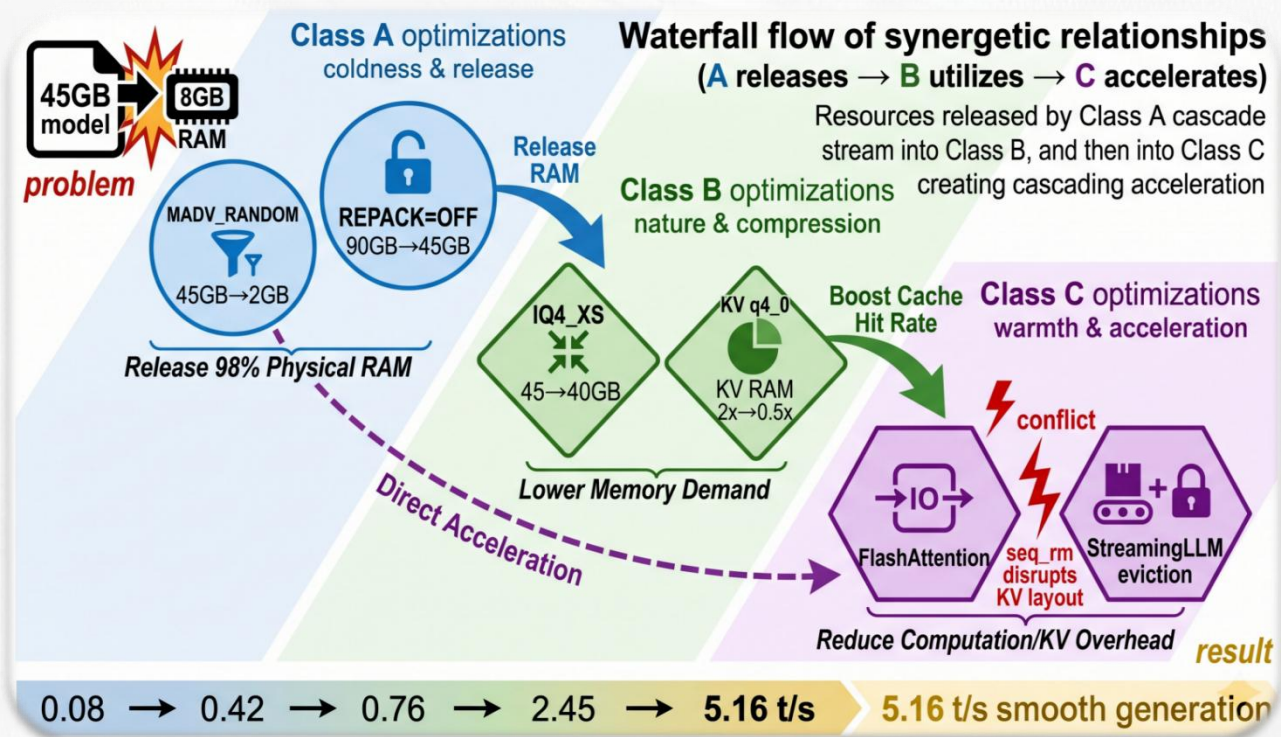
Figure 8 illustrates how PowerInfer coordinates GPU and CPU in processing a layer's neurons. It classifies neurons based on offline data, assigning hot-activated ones (e.g., indices 3, 5, 7) to GPU memory and others to CPU memory. Upon receiving an input, a predictor identifies which neurons in the current layer are likely to be activated. For instance, it predicts activation for neurons 3, 4, and 5. It is crucial to note that hot-activated neurons, identified through offline statistical analysis, may not consistently match the runtime activation behaviors. For example, neuron 7, though labeled as hot-activated, is forecasted to be inactive in this case.

Both CPU and GPU then process predicted active neurons, ignoring inactive ones. The GPU computes neurons 3 and 5, while the CPU handles neuron 4. Once neuron 4's computation is complete, its output is sent to the GPU for result integration.

2 项目简述

我们的方案是什么？

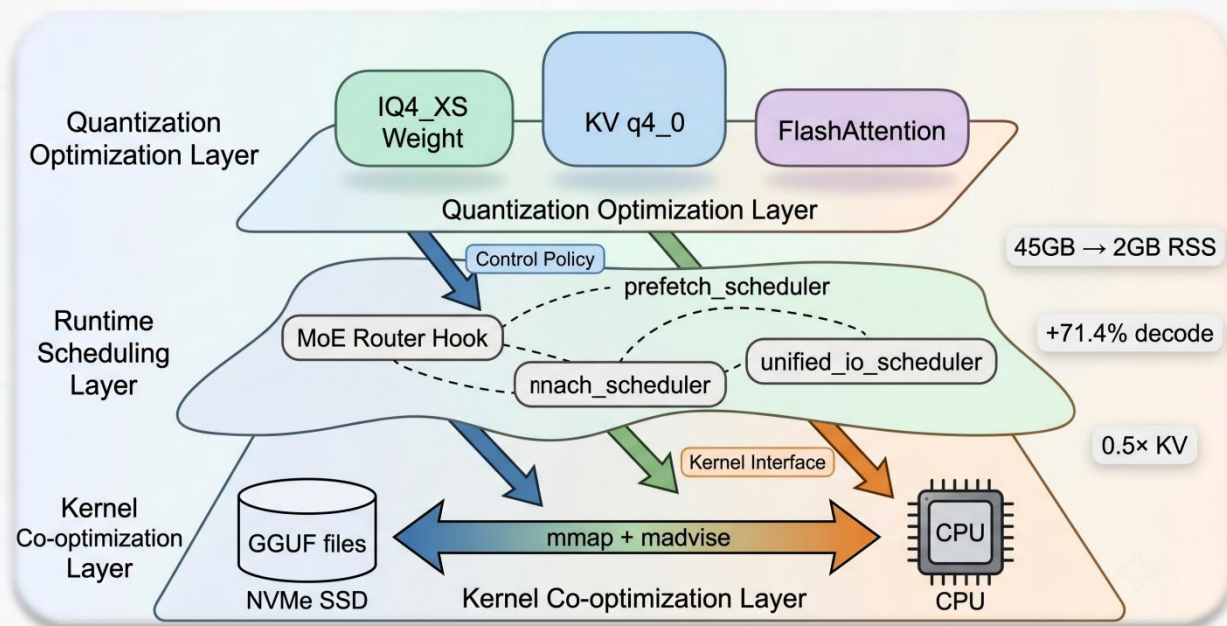
- OS 原生统筹：摒弃零散的单点优化，将**操作系统统虚拟内存机制**作为 MoE 稀疏性的天然接口
- 级联加速 (Cascading Synergy)
 - 内核层释放 (Class A)：干预 OS 缺页与预读调度，释放物理内存底座。
 - 量化层利用 (Class B)：压缩模型体积，吸收内核释放的RAM，提高页缓存命中率。
 - 计算层爆发 (Class C)：缓存红利彻底解除 I/O 封印，支撑融合算子完成算力冲刺。
- 形成“释放 → 利用 → 加速”的核心级联效应



3 核心设计

SLIM-ARC 中的核心优化设计

SLIM-ARC 整体架构 —— 自底向上的三层协同



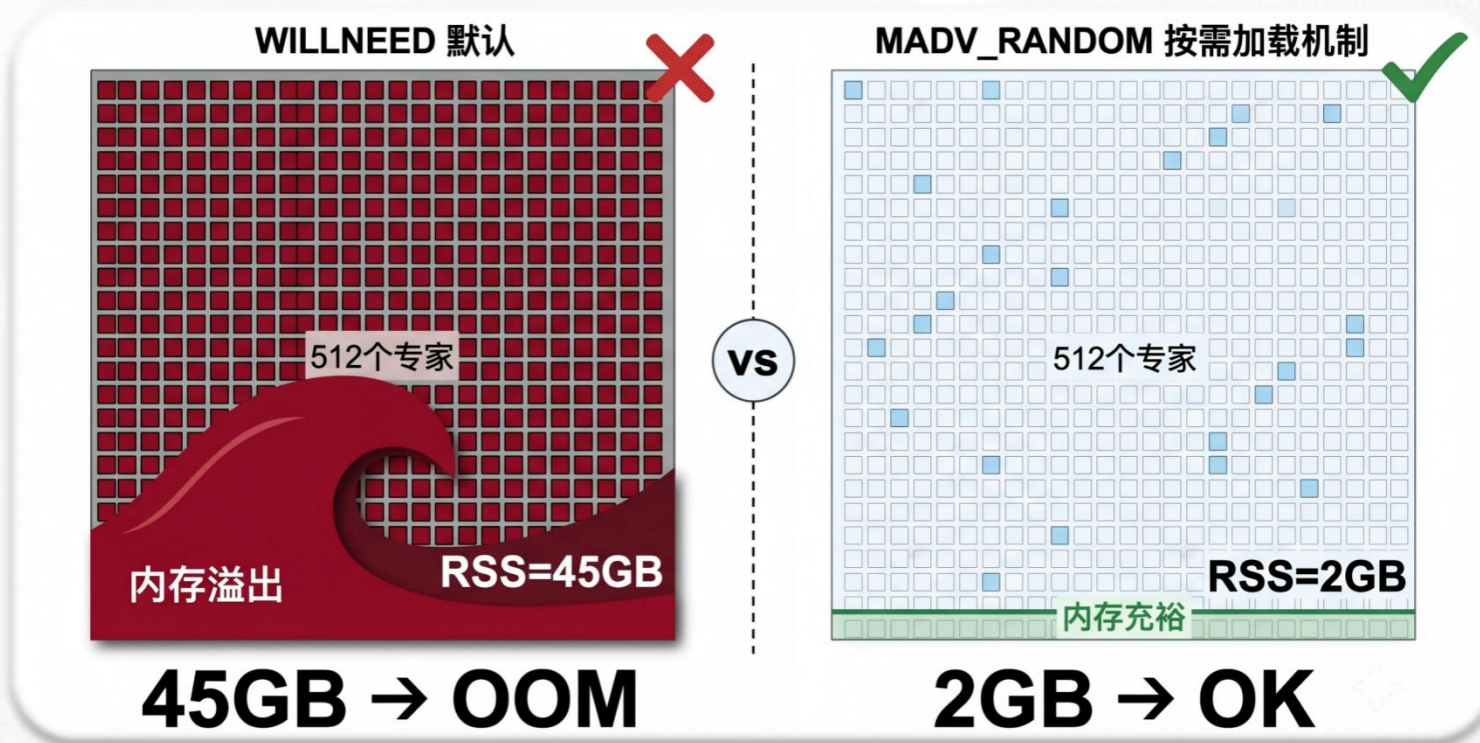
➤ OS原生接口：

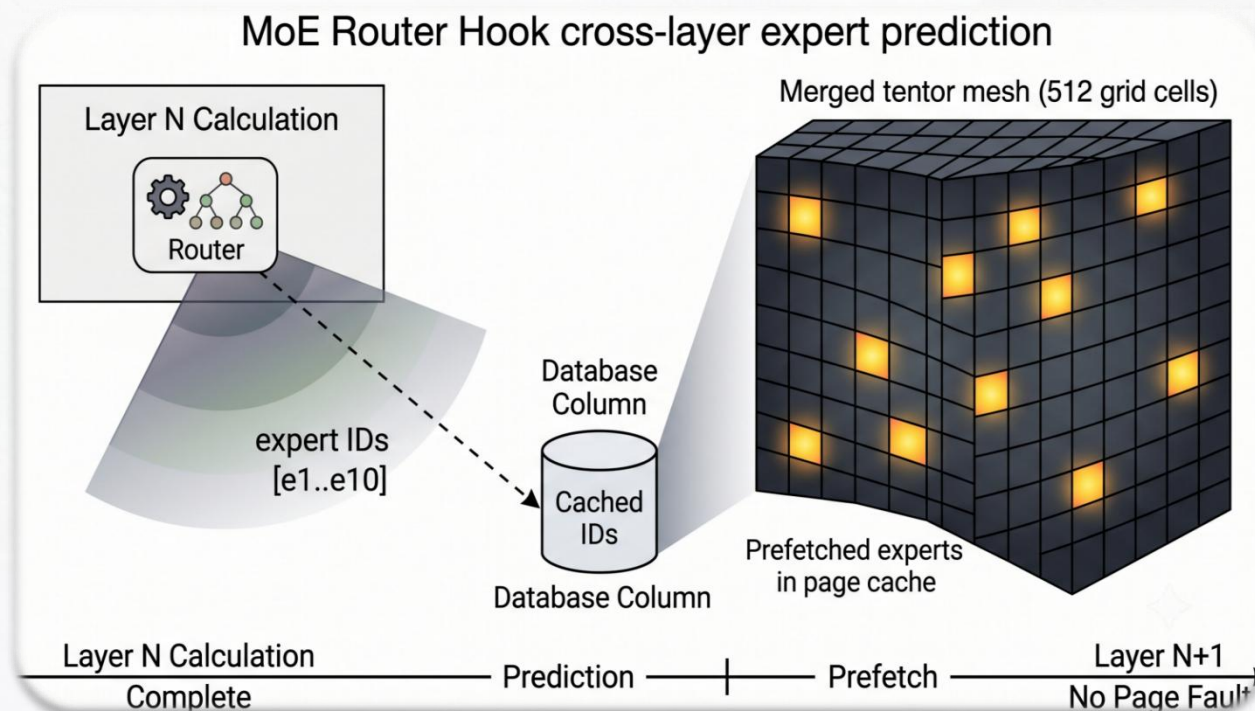
- 摒弃高侵入性的用户态内存重写
- 利用操作系统原生的虚拟内存机制

➤ 内核协同层：

- ***MMAP***将 45GB 模型文件映射至虚拟地址空间
- 标准***POSIX***接口接管内核的缺页与预取行为
- 运行时调度层：基于MoE路由输出与运行时阶段，动态协调权重预取、专家加载与 KV 换页的多路 I/O 竞争。
- 量化优化层：融合极限极低比特量化（IQ4_XS）与 ***FlashAttention***，从算法侧大幅降低绝对内存需求与算力开销。

- 传统预读机制：
 - 触发内核无差别的“顺序预读”
 - 45GB大模型在8GB环境下瞬间引发OOM
- 击穿内存墙的核心指令：
 - 调用 `posix_madvise(MADV_RANDOM)` 彻底禁用内核预读
 - 将内存加载的控制权完全交还给真实的缺页异常 (Page Fault)





➤ 跨层级路由信号捕获:

➤ 精准提取并缓存前一层的 *ffn_moe_topk* 张量

➤ 获取 Router 输出的激活专家 ID

➤ 3D 张量物理偏移计算:

➤ 针对上游框架合并存储的 3D 专家权重

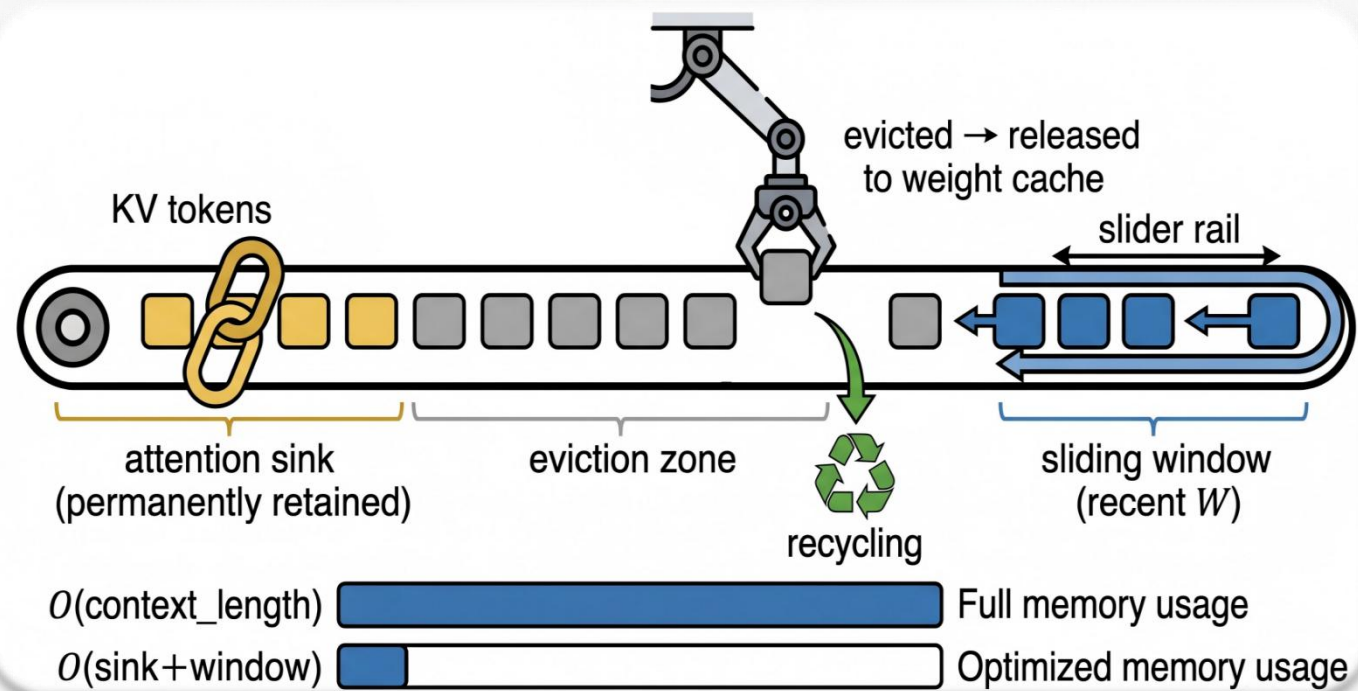
➤ 计算出目标专家所在的连续内存子区域

➤ 提前掩盖 I/O 延迟:

➤ 将提取的专家 ID 用于下一层的激活动态预测, 提前向内核下发选择性预取指令

StreamingLLM 恒定上下文内存驱逐

- 传统长文本场景的内存爆炸：
 - 随着对话增加，KV Cache会呈现**线性膨胀**，最终反超模型权重成为 OOM 的新元凶
- “传送带式”缓存驱逐机制：
 - 引入**StreamingLLM**策略，保留前4 个初始Token (Attention Sink) 以稳定注意力分母，同时仅维护最新生成的滑动窗口
- 空间复杂度降维：
 - 驱逐中间不活跃的旧 Token
 - 将长上下文推理的 KV 内存占用从线性增长约束为**常量规模**



4 工程实现

SLIM-ARC 的具体工程代码实现

环境变量接口

- SLIM_ARC_DISABLE=1 全关，等价 upstream baseline
- SLIM_ARC_NO_MADV_RANDOM=1 只关 MADV_RANDOM
- SLIM_ARC_NO_PREFETCH=1 只关预取调度器
- SLIM_ARC_DYNAMIC_MADV=1 动态切换 prefill/decode

llama.cpp 源文件修改 (4个)

- llama-model-loader.cpp (+65行) : MADV_RANDOM、张量注册
- llama-context.cpp (+85行) : graph_compute 调度、专家ID提取
- llama-kv-cache.cpp (+7行) : clear() 释放 KV 物理页
- CMakeLists.txt (+4行) : 加入构建目标

SLIM-ARC 独立模块

模块	职责
prefetch	层感知预取调度器
unified-scheduler	统一I/O带宽预算调度器
kv-eviction	KV Cache换页管理器
on-demand	早期pread方案 (已禁用)



```
1  madv_block = ""mmaps_used.emplace_back(mapping->size(), 0);
2
3      // SLIM-ARC: MADV_RANDOM for large models (>6GB) to enable demand paging.
4      // Tradeoff: prefill slower (no readahead), decode 3-4x faster (MoE sparsity).
5      // SLIM_ARC_NO_MADV_RANDOM=1 to disable.
6      {
7          bool slim_arc_disabled = getenv("SLIM_ARC_DISABLE") != nullptr;
8          bool no_madv = getenv("SLIM_ARC_NO_MADV_RANDOM") != nullptr;
9          size_t msz = mapping->size();
10         if (!slim_arc_disabled && !no_madv && msz > (6ULL << 30) &&
11             mapping->addr() && msz > 0) {
12             (void) posix_madvise(mapping->addr(), msz, POSIX_MADV_RANDOM);
13             // SLIM-ARC: register mmap region for dynamic MADV switching
14             slim_arc::register_mmap_region(mapping->addr(), msz);
15         }
16     }""
```

➤ 条件触发:

- 仅对 >6GB 的大模型生效
- 小模型走原生路径无开销

➤ cgroup 自适应: 读取

/sys/fs/cgroup/memory.max,

模型 <60% 内存上限则跳过预取

➤ 环境变量开关:

SLIM_ARC_DISABLE /

SLIM_ARC_NO_MADV_RANDOM 支持

消融对比

层感知异步预取 + MoE Router Hook



```
1 void prefetch_scheduler::worker_loop() {
2     while (true) {
3         int target_layer;
4         uint64_t sig;
5         {
6             std::unique_lock<std::mutex> lk(mtx_);
7             cv_.wait(lk, [this] { return stop_ || target_layer_ != current_layer_.load(); });
8             if (stop_) return;
9             target_layer = target_layer_;
10            sig = target_signature_;
11        }
12
13        if (sig != signature_.load()) continue; // stale
14
15        current_layer_.store(target_layer);
16
17        // Prefetch layers [target+1, target+window]
18        size_t bytes_this_round = 0;
19        for (int w = 1; w <= window_; ++w) {
20            int layer = target_layer + w;
21            if (layer < 0 || (size_t)layer >= tensors_by_layer_.size()) continue;
22            for (const auto & t : tensors_by_layer_[layer]) {
23                if (t.addr == nullptr || t.size == 0) continue;
24                // posix_madvise is thread-safe and idempotent
25                (void) posix_madvise(t.addr, t.size, POSIX_MADV_WILLNEED);
26                bytes_this_round += t.size;
27            }
28        }
29
30        total_bytes_.fetch_add(bytes_this_round);
31        total_calls_.fetch_add(1);
32    }
33 }
```

```
1 if (status == GGML_STATUS_SUCCESS) {
2     if (auto * s = slim_arc::get_global_prefetch_scheduler()) {
3         int n_nodes = ggml_graph_n_nodes(gf);
4         for (int i = 0; i < n_nodes; ++i) {
5             struct ggml_tensor * t = ggml_graph_node(gf, i);
6             if (!t || !t->name) continue;
7             if (strstr(t->name, "ffn_moe_topk") != nullptr && t->data != nullptr) {
8                 int layer = slim_arc::tensor_layer_from_name(t->name);
9                 if (layer < 0) continue;
10                int n_expert_used = (int) t->ne[0];
11                if (n_expert_used <= 0) continue;
12                const int32_t * ed = (const int32_t *) t->data;
13                std::vector<int> ue;
14                for (int e = 0; e < n_expert_used && e < 64; ++e) {
15                    int eid = (int) ed[e];
16                    if (eid >= 0) {
17                        bool found = false;
18                        for (int x : ue) if (x == eid) { found = true; break; }
19                        if (!found) ue.push_back(eid);
20                    }
21                }
22                if (!ue.empty()) s->cache_router_experts(layer, ue.data(), (int) ue.size());
23                break;
24            }
25        }
26    }
27 }
```

➤ 按层注册权重，前看三层预取。

➤ 提取路由专家，跨层精准预读。

静态分配矩阵

```
1 static constexpr double WEIGHT_RATIOS[5][3] =  
2 { // {weight, kv, expert}  
3   {0.60, 0.10, 0.30}, // PREFILL_SHORT  
4   {0.50, 0.20, 0.30}, // PREFILL_LONG  
5   {0.70, 0.20, 0.10}, // DECODE_SHORT  
6   {0.30, 0.60, 0.10}, // DECODE_LONG  
7   {0.20, 0.20, 0.60}, // MOE_DECODE  
8 };
```

➤ 将权重预取、KV 换页、MoE 专家预取三路 I/O 纳入统一带宽预算。

➤ 5 阶段 $\times$ 3 路 I/O = 15 维静态预算表 + 运行时三元反馈闭环。

动态自适应修正

```
1 if (current_stats_.weight_stalls > 0) {  
2   // Weight prefetch is stalling compute → increase weight budget  
3   double adjustment = std::min(0.1, current_stats_.weight_stalls * 0.02);  
4   budget.weight_bytes = (size_t)(budget.weight_bytes * (1.0 + adjustment));  
5   budget.expert_bytes = (size_t)(budget.expert_bytes * (1.0 - adjustment * 0.  
6 5)); budget.kv_bytes = (size_t)(budget.kv_bytes * (1.0 - adjustment * 0.5));  
7 }
```

```
1 kv_evict_block = ""
2 // SLIM-ARC Phase 4: StreamingLLM-style KV eviction after decode.
3 // Keeps first N tokens (attention sinks) + last W tokens (sliding window),
4 // evicts tokens in between via llama_memory_seq_rm to cap KV memory.
5 // Enabled via SLIM_ARC_KV_EVICT=1, configured by SLIM_ARC_KV_SINK/WINDOW.
6 if (status == GGML_STATUS_SUCCESS && !batched) {
7     static bool kv_evict_enabled = getenv("SLIM_ARC_KV_EVICT") != nullptr;
8     if (kv_evict_enabled) {
9         static int kv_sink = []{ const char * e = getenv("SLIM_ARC_KV_SINK"); return e ? atoi(e) : 4; }();
10        static int kv_window = []{ const char * e = getenv("SLIM_ARC_KV_WINDOW"); return e ? atoi(e) : 1024; }();
11        static int last_evicted_pos = 0;
12        static bool first_call = true;
13        if (first_call) {
14            fprintf(stderr, "SLIM-ARC KV eviction: ENABLED (sink=%d, window=%d)\n", kv_sink, kv_window);
15            first_call = false;
16        }
17        llama_pos pos_max = memory->seq_pos_max(0);
18        int seq_len = (int) pos_max + 1;
19        int threshold = kv_sink + kv_window;
20        if (seq_len > threshold) {
21            int p0 = (last_evicted_pos > kv_sink) ? last_evicted_pos : kv_sink;
22            int p1 = seq_len - kv_window;
23            if (p1 > p0) {
24                memory->seq_rm(0, p0, p1);
25                last_evicted_pos = p1;
26                fprintf(stderr, "SLIM-ARC KV eviction: evicted [%d,%d] (seq_len %d->%d)\n",
27                    p0, p1, seq_len, seq_len - (p1 - p0));
28            }
29        }
30    }
31 }
```

- sink token 稳定注意力 + 滑动窗口控制 KV 规模, decode +9.6%, KV 内存占用从 $O(L)$ 降至 $O(1)$
- KV Cache clear() 时调用 `posix_madvise(DONTNEED)` 主动归还物理页 (scripts/apply-slim-arc.py:345-356), 避免内存碎片累积

5 测试与评估

SLIM-ARC带来了哪些提升？

实验环境配置 (表7)

配置项	规格
CPU	i9-13900H (6P+8E, 20T)
RAM	32 GB DDR5 (WSL2)
存储	NVMe SSD (~3.5 GB/s)
指令集	AVX2 + AVX_VNNI + FMA
框架	llama.cpp + SLIM-ARC

测试模型规格 (表8)

模型	量化	大小	专家
Qwen3-4B	Q4_K_M	2.4 GB	—
OLMoE-1B-7B	Q4_K_M	3.9 GB	64
Qwen3-Next-80B	IQ4_XS	39.7 GB	512

测试方法与基线

- 串行执行, 每次前 drop_caches 清空 page cache (冷启动)
- llama-bench 测量 pp/tg 吞吐量, 重复3次取均值
- 精度: GSM8K 8-shot + WikiText-2 PPL (ALEM协议)
- 基线: upstream llama.cpp 默认配置 (mmap+WILLNEED+KV f16)
- 消融: 完整 SLIM-ARC 基础上逐个关闭优化

IQ4_XS 量化三档

环境(线程)	pp	tg	说明
8 GB (4核)	—	—	冷启动超时
16 GB (8核)	—	2.27 t/s	勉强可用
32 GB (8核)	4.44	3.03 t/s	流畅交互

Q4_K_M 量化三档

环境(线程)	pp	tg	说明
8 GB (4核)	—	—	冷启动超时
16 GB (8核)	1.96	—	tg 超时
32 GB (8核)	—	2.68 t/s	比IQ4_XS低12%

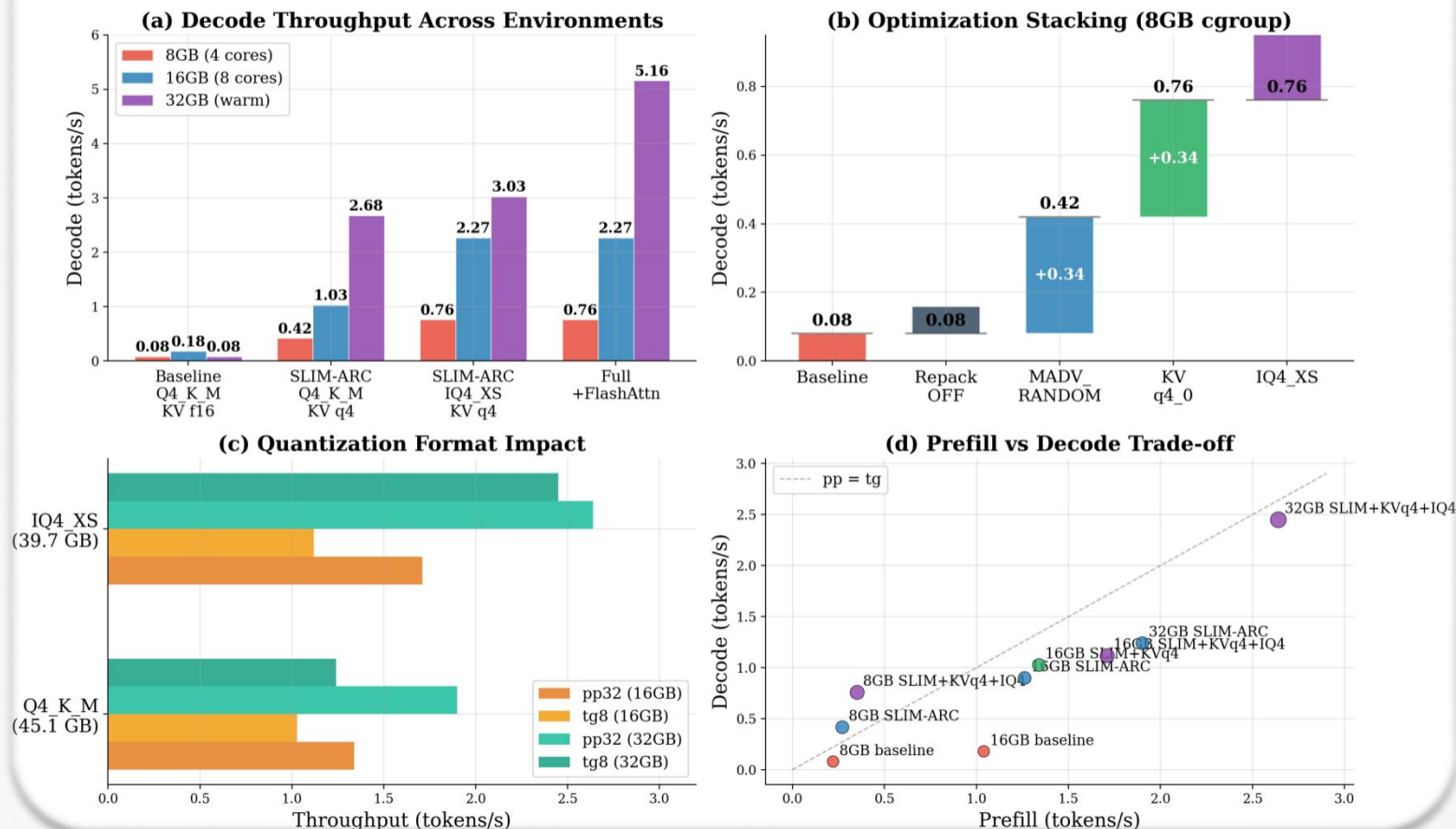
关键发现

- 8GB: 40 - 45GB 模型触发大量 page fault, 长序列冷启动超时 (短序列 0.76 t/s, 9.5×)
- 16GB: 2.27 t/s, **80B 端侧首次可用** (6.2×)
- 32GB: 3.03 t/s 流畅交互, IQ4_XS 比 Q4_K_M 快约 **13%**

环境越受限，相对收益越大 ——
SLIM-ARC 核心价值所在

性能全景：三档环境与优化堆叠

SLIM-ARC Performance Landscape: Qwen3-Next-80B



全景洞察

- 端到端累计 **64.5X** (8GB baseline 0.08 → 32GB 全优化 5.16, 跨环境)
- 32GB 内 FlashAttention **+71.4%** (3.01→5.16), 计算融合成新瓶颈
- 环境越受限, 相对收益越大

8GB→32GB: 从不可用到流畅,
SLIM-ARC 跨越内存墙

小模型 2GB+1核极端环境

模型	大小	tg16	说明
Qwen3-4B	2.4 GB	2.58	可运行
OLMoE-1B-7B	3.9 GB	8.92	流畅

GSM8K 推理精度

模型	量化	GSM8K	t/s
Qwen3-4B	Q4_K_M	75%	7.8
80B	Q4_K_M+KVq4	50%	0.7
80B	IQ4_XS+KVq4	0%	1.7

关键发现

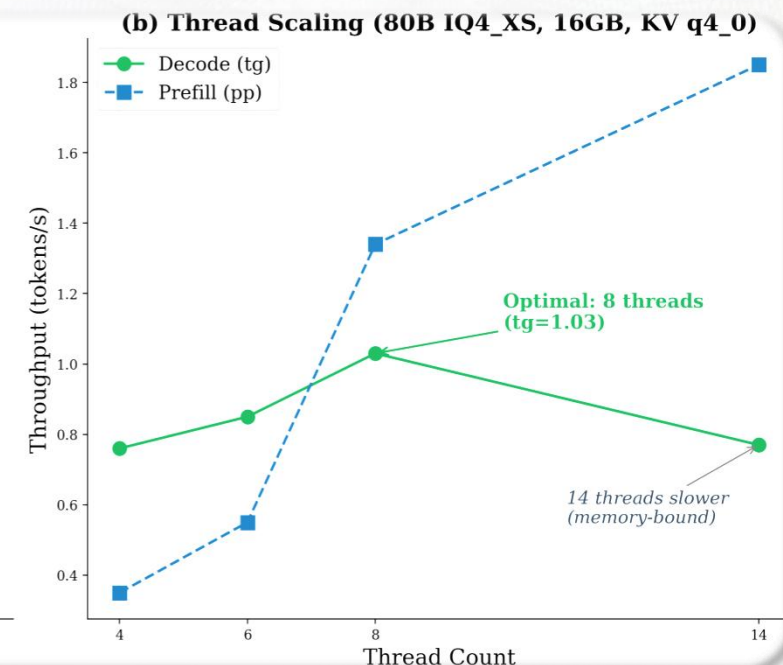
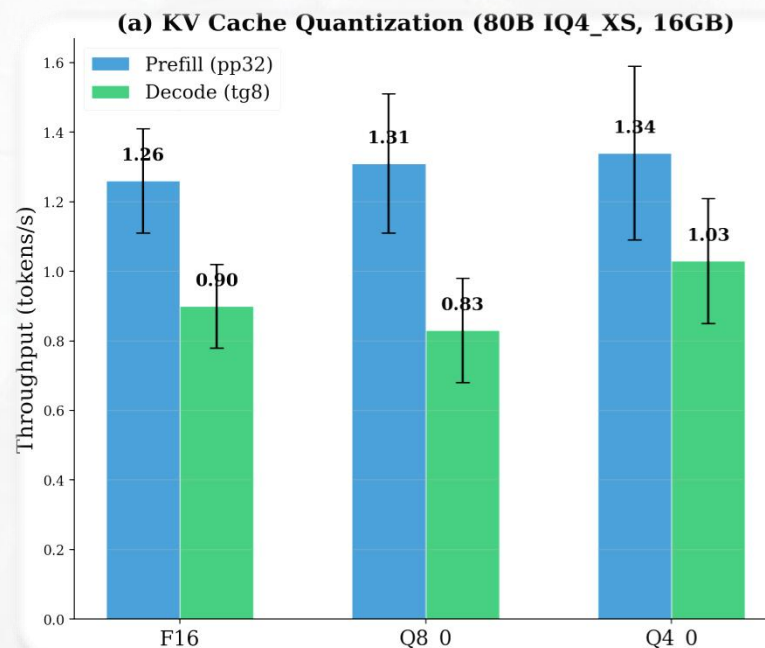
- OLMoE 凭借 MoE 稀疏性，在 2GB+1核极端环境达 8.92 t/s，验证 MADV_RANDOM 对小模型同样有效
- IQ4_XS 较 Q4_K_M 仅减 0.25 bit，GSM8K 精度由 50% 骤降至 0%（非线性崩塌）；典型失败为"计算正确、答案写错"
- PPL 不足以反映 MoE 推理能力，GSM8K 是更敏感的精度代理指标

KV 量化类型对比

KV 类型	pp64	tg48	KV 内存	vs Q4_0
F16	—	3.92	2×	+4%
Q8_0	5.37	2.91	1×	-23%
Q4_0	4.44	3.03	0.5×	best

KV 量化要点

- Q4_0 使 KV 内存减半 (**6GB→3GB**)
- 32GB 冷启动下略慢于 F16 (+4%)
- 8GB 受限环境 Q4_0 带来 **+14%** 提升
- 优化的环境相关性：内存充裕略负，受限正向



线程扩展性

- 8 线程为最优配置
- 更多线程因内存带宽饱和无增益
- decode 为 memory-bound，受限于 NVMe 带宽

消融实验：单点消融定位各组件贡献

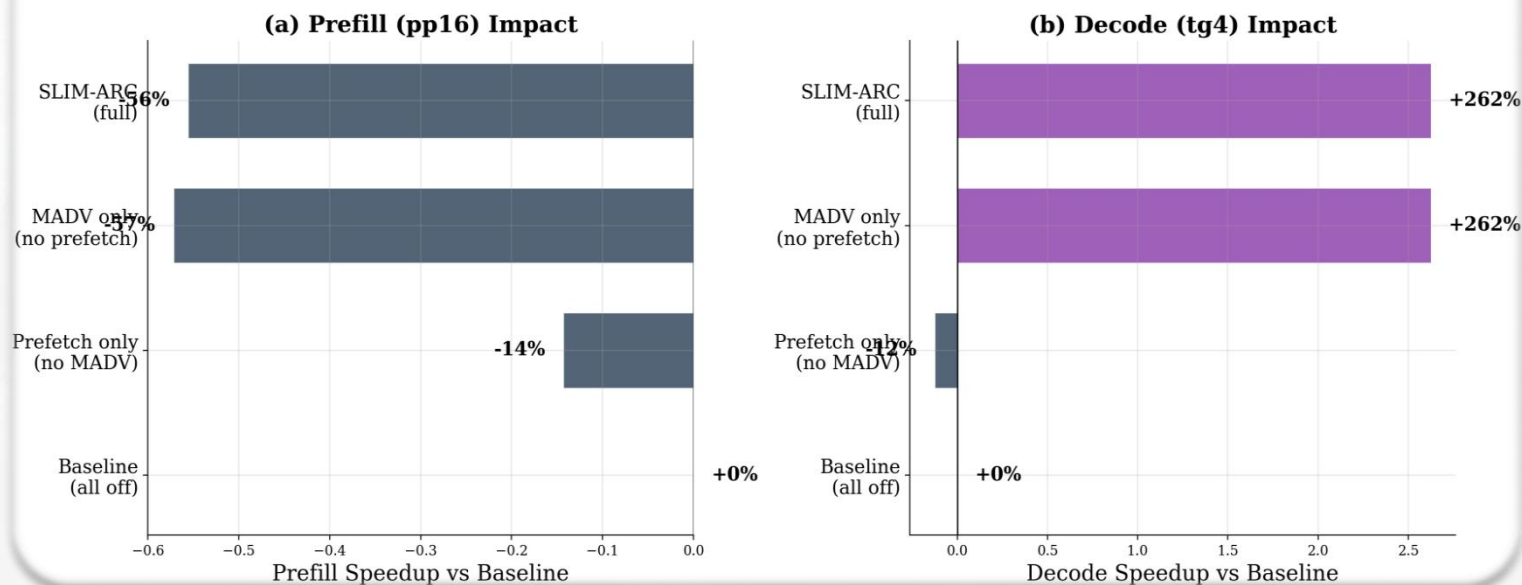
单点消融数据

配置	pp64	tg48	vs Full	贡献
Full SLIM-ARC	4.44	3.03	—	基线
-KV q4_0 (f16)	—	3.92	+4%	冷启动略负
-MADV (DISABLE)	3.72	2.15	-41%	最大贡献
+Eviction	—	3.30	-12%	与FA冲突

结论

- MADV_RANDOM 是 decode 加速最大贡献者 **(-41%)**
- 物理内存节省机制：Qwen3-Next-80B 每 token 仅激活 512 个专家中的 10 个 **(2%)**

Four-Way Ablation: Component Contribution Analysis (80B Q4_K_M, 8GB)



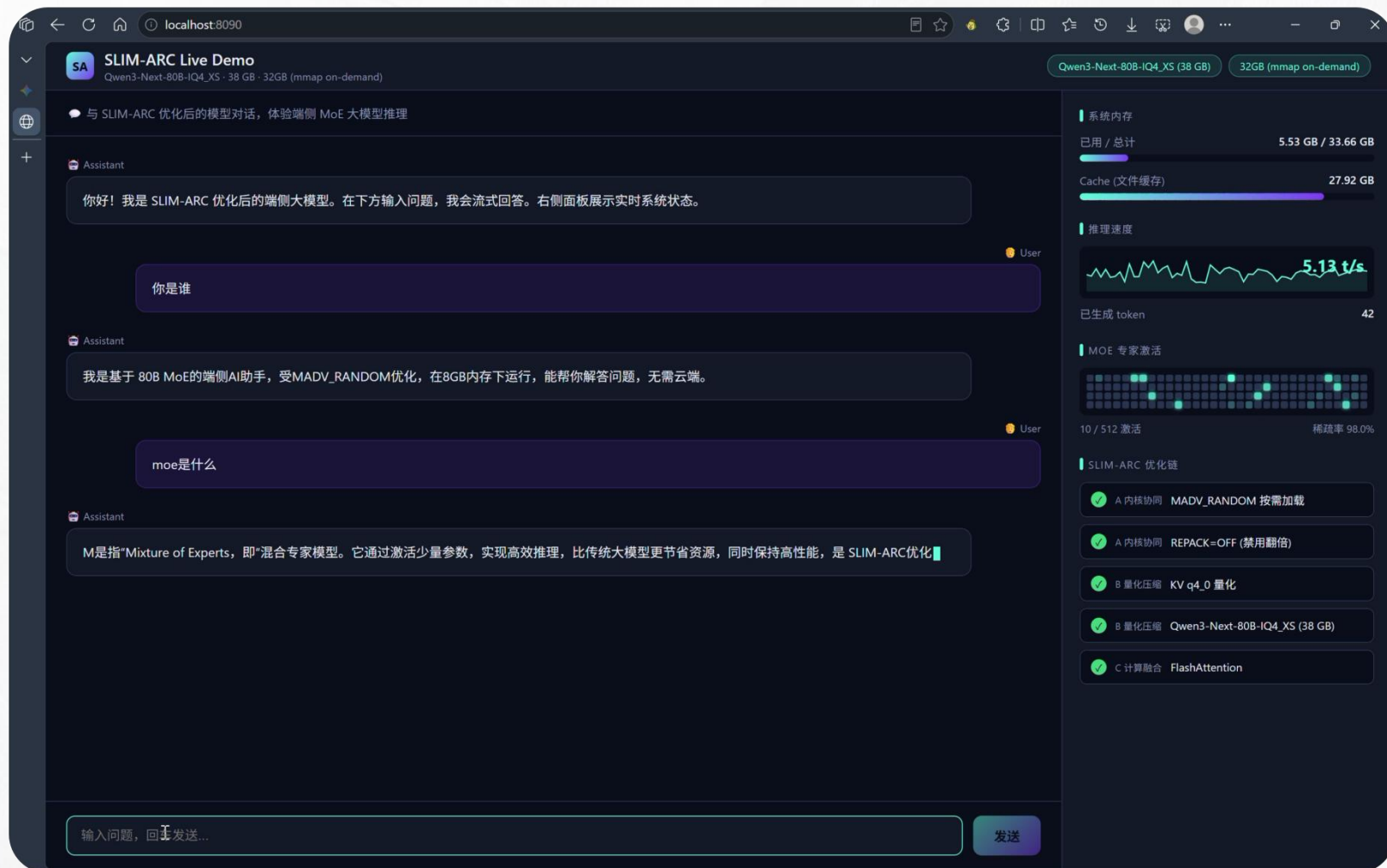
协同与冲突

- MADV_RANDOM $\Leftrightarrow$ KV q4_0 (协同促进)
- Eviction VS FlashAttention (机制冲突)

6 总结与展望

SLIM-ARC 下一站是哪里？

实际效果 Live Demo



The screenshot displays the SLIM-ARC Live Demo interface. The main chat area shows a conversation between a User and an Assistant. The Assistant's responses are as follows:

- Assistant: 你好！我是 SLIM-ARC 优化后的端侧大模型。在下方输入问题，我会流式回答。右侧面板展示实时系统状态。
- Assistant: 我是基于 80B MoE的端侧AI助手，受MADV_RANDOM优化，在8GB内存下运行，能帮你解答问题，无需云端。
- Assistant: M是指“Mixture of Experts”，即“混合专家模型”。它通过激活少量参数，实现高效推理，比传统大模型更节省资源，同时保持高性能，是 SLIM-ARC优化

The right sidebar provides real-time system status:

- 系统内存**: 已用 / 总计 5.53 GB / 33.66 GB
- Cache (文件缓存)**: 27.92 GB
- 推理速度**: 5.13 t/s
- 已生成 token**: 42
- MOE 专家激活**: 10 / 512 激活, 稀疏率 98.0%
- SLIM-ARC 优化链**:
 - ✓ A 内核协同 MADV_RANDOM 按需加载
 - ✓ A 内核协同 REPACK=OFF (禁用翻倍)
 - ✓ B 量化压缩 KV q4_0 量化
 - ✓ B 量化压缩 Qwen3-Next-80B-1Q4_XS (38 GB)
 - ✓ C 计算融合 FlashAttention

点击按钮
观看视频

核心贡献

- 贡献一：发现并验证 MADV_RANDOM 对 MoE decode 的核心加速效应

消融实验证实 MADV_RANDOM 为 decode 提升唯一驱动——关闭后 tg48 由 3.03 降至 2.15 t/s (-29%)，是全栈贡献最大的单点优化。

- 贡献二：识别并消除 GGML_CPU_REPACK 内存翻倍缺陷

构建时关闭 repack (-DGGML_CPU_REPACK=OFF)，CPU backend 直接基于 mmap 原始权重计算。80B 模型从 OOM 转为可运行，且热缓存场景下 decode 无性能损耗。

- 贡献三：构建端到端内存感知优化栈

消融揭示 A 类优化 (MADV 释放内存) 为 B 类 (KV q4_0 / IQ4_XS) 创造生效条件，体现"A 释放→B 利用→C 加速"的级联效应，超越单点优化之和。

- 贡献四：实现 80B MoE 模型在受限环境的可用性突破

在 32 GB 热缓存环境下，IQ4_XS 量化的 Qwen3-Next-

80B 达到了 5.16 tokens/s 的 decode 速度 (含 FlashAttention)，实现了流畅的交互式推理。在 8 GB 最受限环境下，达到了 0.76 tokens/s，相比 baseline 提升 850% (9.5 倍)。

决赛探索方向：

- 超参数机器学习联调：贝叶斯优化/网格搜索自动搜索最优组合
- KV eviction 深度集成：基于注意力分数的 Block 级驱逐 + 跨层投机预取
- 更激进量化：Q2_K/IQ2_XXS (~22 - 28GB) 彻底消除 8GB 数据波动
- Tile 级微流水线：权重切分对齐 L2/L3 cache, I/O 与计算重叠
- 统一调度器协同验证：权重预取+KV换页+专家预取三路竞争
- 端侧 Agent 语义缓存：edge-cloud 协同, 84% hit rate 上下文摘要
- OS 底层方案：userfaultfd 批量缺页、readahead 精确控制、内核模块 DMA 预读



请各位评委老师批评指正！

Email: ouyyp5@mail2.sysu.edu.cn

Website: <https://slim.nexa-lang.com/>

<https://www.bilibili.com/video/BV1fXTF6HEAw/>